

DESIGN & BUILD TUTORIAL

uNode32U4 Bare-Chip USB Dev Board

A complete, manufacturable 2-layer board built around one bare ATmega32U4 and thirty-five primary components — power integrity, crystal clock, reset & bootloader, USB routing, and PCB layout craft — designed end-to-end in tscircuit, where circuits are React components.

Designed with tscircuit · React for electronics

Verified netlist · 35 components · 105 routed traces · October 2026

ATMEGA32U4 · TQFP-44 · USB 2.0 · LEONARDO-COMPATIBLE

Table of Contents

1. The Bare-Chip Philosophy	2
2. tscircuit: React for Circuits	3
3. The ATmega32U4 and the Verified Pin Map	5
4. Power Architecture	7
4.1 The front end: fuse, and the 22Ω pair	7
4.2 Decoupling: one capacitor per pin, and why	8
5. The Clock: Crystal Theory and Load-Capacitance Math	9
5.1 The load-capacitance equation, worked	9
5.2 Layout rules the framework enforces for you	10
6. Reset, Bootloader and ICSP	11
6.1 The reset network	11
6.2 BOOT: the hardware bootloader pin	11
6.3 Fuses, bit by bit	11
6.4 ICSP: the umbilical port	12
7. Indicators, I ² C and the Breakout Headers	12
7.1 Four LEDs, three calculations	12
7.2 I ² C pull-ups	13
7.3 The headers	13
8. PCB Layout and Routing Craft	14
8.1 Placement strategy	14
8.2 The routing pass — and its honest report	14
8.3 Layout rules worth internalizing	15
8.4 From code to factory	15
9. Bill of Materials	16
10. Bring-Up, First Firmware and Troubleshooting	17
10.1 Assembly order	17
10.2 First power	17
10.3 Bootloader and first sketch	17
10.4 Troubleshooting	18
10.5 Where to go next	18

1. The Bare-Chip Philosophy

Almost every electronics curriculum stalls at the same place: you blink LEDs on a pre-built dev board, and then nothing. The board works, but you never learn why. Between the USB cable and your blinking LED sit a dozen deliberate engineering decisions — a fuse, a crystal with carefully chosen load capacitors, a reset network, a boot strategy, a decoupling capacitor at every power pin — and a pre-built board hides every one of them under a friendly silk-screened logo. This tutorial is about taking all of that back.

The **uNode32U4** project designs a complete, manufacturable USB dev board around a **single bare ATmega32U4** microcontroller in a TQFP-44 package — the same raw silicon Arduino buys to manufacture the Leonardo — plus about thirty-five primary components: resistors, capacitors, a crystal, diodes, LEDs, buttons, fuses and headers. No Arduino module, no breakout board, no hidden firmware. You will wire every section yourself, understand every value, and end with a board you can order from any fab, solder in an evening, and program with the standard Arduino toolchain.

The design vehicle is **tscircuit**, an open-source framework that describes electronics as React components. Instead of drawing a schematic, you write TypeScript; instead of pushing copper in a layout editor, you place components with coordinates and let the built-in autorouter handle the wiring, then inspect and refine. The full project accompanying this PDF is runnable: `tsci dev` opens a live 3-D view of the board, and `tsci export` produces the gerbers a factory accepts.

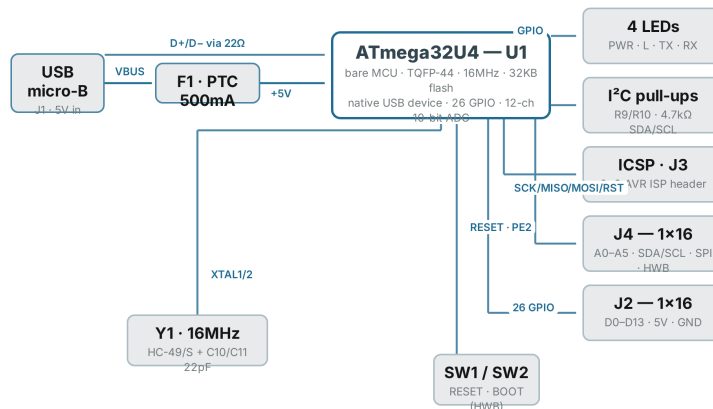


Figure 1-1 — System block diagram. One bare MCU, all support circuitry explicit. Every block in this diagram becomes a commented section of the project's `index.tsx`.

What makes this particular circuit a good teacher is that it is *complete but not trivial*. It exercises the full set of skills a real board designer uses daily: a protected power entry, a precision analog supply branch, a crystal with load-capacitance math, a reset and bootloader strategy, a USB differential pair, firmware-driven

indicators, and a disciplined PCB layout — all on a cheap 2-layer board. Master this board and the jump to your own custom MCU design is a change of part number, not a change of method.

35 components, all primary parts + one bare MCU	44 TQFP pins mapped and cross-verified against 3 sources	2 layer PCB, 72 × 50 mm, JLCPCB-friendly
---	--	--

2. tscircuit: React for Circuits

tscircuit's core idea is disarmingly simple: a circuit is a component tree, and React is the world's best-tested system for describing component trees. Where a web app renders elements into a DOM, tscircuit renders and elements into *Circuit JSON* — a complete, vendor-neutral description of your schematic, PCB and 3-D model. The same React Fiber machinery that powers React Native and react-three-fiber drives the render, so everything you already know about components, props and composition transfers directly.

A circuit lives inside a element. Every physical part is a JSX element with a *footprint* (the copper pattern it solder onto), a position on the board, and electrical properties. Wiring is expressed with *selectors* — CSS-like strings that name ports — so a trace reads almost like a sentence:

index.tsx — the shape of every tscircuit project

```
<board width="72mm" height="50mm">
  <chip name="U1" footprint="tqfp44" pinLabels={{ pin1: "PE6", pin44: "AVCC" }} />
  <resistor name="R3" resistance="10k" footprint="0603" pcbX={2} pcbY={-3} />
  <trace from=".R3 > .left" to="net.V5" />
  <trace from=".R3 > .right" to=".U1 > .RESET" />
</board>
```

Three conventions carry ninety percent of the framework. **Footprinter strings** generate standard land patterns from a short code: "tqfp44" expands to a 44-pad, 0.5 mm-pitch TQFP with a 3-D model, "0603" to a passive pad pair, "hc49" to a crystal can. **Pin labels** turn anonymous pin numbers into named ports, so pin 44 becomes `.U1 > .AVCC`. And **nets** are created on demand: any trace that ends at `net.GND` joins every other trace with the same target, exactly like a net label in KiCad. Point-to-point wires are direct traces; anything touching three or more places gets a named net.

The tooling loop is a web-dev loop. Run `tsci dev` and a local server hot-reloads the PCB, schematic and 3-D views every time you save. Type-check with plain `tsc`. Export gerbers, pick-and-place and BOM with `tsci export` when you are ready to manufacture. Because circuits are code, they diff cleanly in git, review in pull requests, and regenerate deterministically — properties no schematic file has ever had.

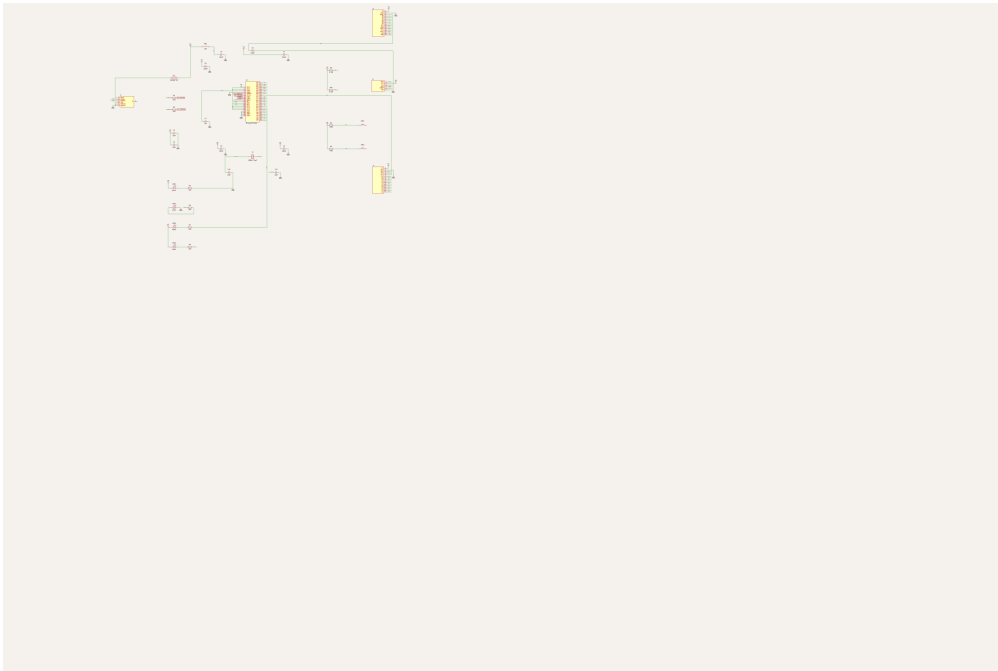


Figure 2-1 — The auto-generated schematic view of the finished board (rendered headlessly from the same code that produces the PCB). Re-flow and re-arrange it interactively in tsci dev.

The `µNode32U4` project is organized the way a real design is sheeted: a `lib/atmega32u4.tsx` component defines the bare chip once (footprint, all 44 pin names, schematic pin arrangement), `lib/usb-micro-b.tsx` hand-crafts a USB connector footprint from primitive pads, and `index.tsx` is the board itself — one flat netlist organized into banner-commented sections that mirror this tutorial's chapters. A single-file copy in `standalone/` pastes straight into the web editor at snippets.tscircuit.com.

lib/atmega32u4.tsx — a reusable 'bare chip' component

```
export const Atmega32U4 = (props: Atmega32U4Props) => (
  <chip
    footprint="tqfp44" // generated land pattern
    pinLabels={ATMEGA32U4_PINOUT} // pin1:"PE6" ... pin44:"AVCC"
    manufacturerPartNumber="ATmega32U4-AU"
    schPinArrangement={
      // readable schematic box:
      leftSide: { direction: "top-to-bottom",
        pins: ["VBUS", "UVCC", "UCAP", "UGND", "DMINUS", "DPLUS",
          "XTAL1", "XTAL2", "RESET", "PE2", "AREF", "AVCC",
          "VCC1", "VCC2", "VCC3", "GND1", "GND2", "GND3", "GND4"] },
      rightSide: { direction: "top-to-bottom",
        pins: ["PE6", "PB0", "PB1", "PB2", "PB3", "PB4", "PB5", "PB6", "PB7",
          "PC6", "PC7", "PD0", "PD1", "PD2", "PD3", "PD4", "PD5",
          "PD6", "PD7", "PF0", "PF1", "PF4", "PF5", "PF6", "PF7"] },
    }
  >
    { ...props }
  </chip>
)
```

3. The ATmega32U4 and the Verified Pin Map

Why the ATmega32U4? Because it is the last AVR that still earns its place on a learning bench in 2026. It carries a **native USB 2.0 full-speed device controller** — no USB-serial bridge chip, no driver gymnastics — alongside 32 KB flash, 2.5 KB RAM, twelve 10-bit ADC channels, four timers with PWM, hardware I²C and SPI, and 26 GPIO. Its Arduino Leonardo lineage means a decade of tutorials, libraries and bootloader support apply to your board the moment it powers up. And because it is an old, well-documented part, the 438-page datasheet answers every question this design asks.

Designing with a bare chip begins and ends with the **pin map**. The TQFP-44 package numbers pins counter-clockwise from the top-left corner: pins 1–11 run down the left edge, 12–22 across the bottom, 23–33 up the right, and 34–44 back along the top. tscircuit's tqfp44 footprint numbers pads in exactly the same order, so datasheet pin numbers can be used directly in code — a detail this project verified empirically before trusting it, by rendering a bare chip and measuring every pad position against the datasheet's Figure 1-1.

Pin	Name	Arduino	Role on this board
1	PE6	D7	GPIO / user interrupt (INT6)
2	UVCC	n/a	USB pad supply → +5V, decoupled
3	D−	n/a	USB D− via 22Ω R2
4	D+	n/a	USB D+ via 22Ω R1
5	UGND	n/a	USB pad ground
6	UCAP	n/a	Internal 3.3V regulator out → 1μF C7
7	VBUS	n/a	VBUS monitor → +5V rail
8	PB0	D17/SS	RX LED (active-low) + SPI SS
9	PB1	D15/SCK	ICSP SCK
10	PB2	D16/MOSI	ICSP MOSI
11	PB3	D14/MISO	ICSP MISO
12	PB7	D11*	PWM GPIO
13	RESET	n/a	10kΩ pull-up + SW1
14	VCC1	n/a	+5V + 100nF C3 at pin
15	GND1	n/a	Ground
16	XTAL2	n/a	Y1 + 22pF C11
17	XTAL1	n/a	Y1 + 22pF C10
18	PD0	D3/SCL	I ² C clock + 4.7kΩ pull-up
19	PD1	D2/SDA	I ² C data + 4.7kΩ pull-up

Pin	Name	Arduino	Role on this board
20	PD2	D0/RX	GPIO (UART1 RX)
21	PD3	D1/TX	GPIO (UART1 TX)
22	PD5	n/a	TX LED (active-low)

Table 3-1 — Pin map, left and bottom edges (pins 1–22). * PWM-capable. All 44 names match the Microchip datasheet (Atmel-7766J).

Pin	Name	Arduino	Role on this board
23	GND2	n/a	Ground
24	VCC2	n/a	+5V + 100nF C4 at pin
25	PD4	D4/A6	GPIO / ADC8
26	PD6	D12/A11	GPIO / ADC9
27	PD7	D6*/A7	PWM GPIO / ADC10
28	PB4	D8/A8	GPIO / ADC11
29	PB5	D9*/A9	PWM GPIO / ADC12
30	PB6	D10*/A10	PWM GPIO / ADC13
31	PC6	D5*	PWM GPIO
32	PC7	D13	'L' LED (active-high) + header
33	PE2	HWB	Boot pin: 10kΩ pull-up + SW2
34	VCC3	n/a	+5V + 100nF C5 at pin
35	GND3	n/a	Ground
36	PF7	A0/TDI	Analog in
37	PF6	A1/TDO	Analog in
38	PF5	A2/TMS	Analog in
39	PF4	A3/TCK	Analog in
40	PF1	A4	Analog in
41	PF0	A5	Analog in
42	AREF	n/a	ADC reference + 100nF C9
43	GND4	n/a	Ground
44	AVCC	n/a	Analog supply via ferrite bead FB1 + 100nF C6

Table 3-2 — Pin map, right and top edges (pins 23–44).

A note on verification, because it matters: this pin table was cross-checked against three independent sources — the pinout figure of Microchip's official datasheet (reconstructed positionally from the PDF's own text coordinates), the official Arduino Leonardo schematic, and the `pins_arduino.h` variant file the Arduino core actually compiles against. Three sources, one answer. When you design with a bare chip, that is

the level of paranoia that pays for itself the first time a board comes back with swapped power pins.

Two quirks of this package are worth internalizing early. First, the 32U4's USB pins (2–7) all live on the *upper-left* edge, which is why this board puts the USB connector on its left edge — the shortest possible route for the most signal-integrity-sensitive wires. Second, PF2 and PF3 do not exist on this package at all; Port F simply has six bonded pins, which is why the analog bank runs A0, A1, A2, A3, then A4, A5 — a mapping inherited directly by the Arduino pin names.

4. Power Architecture

Power is where bare-chip boards live or die, so this design treats the +5V rail as an engineered system rather than a wire. The chain is deliberately conventional — convention is what makes it robust: USB VBUS enters through a resettable fuse, becomes the +5V rail (net.V5), and distributes to four separate supply pins on the MCU, each with its own local energy store, plus a filtered branch for the analog supply.

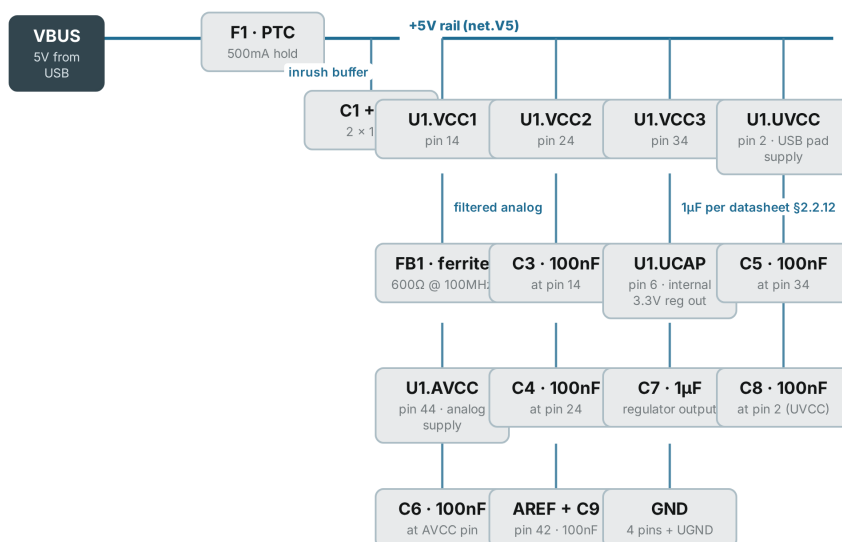


Figure 4-1 — The +5V distribution tree. Every leaf has its own local capacitor; the analog branch is isolated behind a ferrite bead.

4.1 The front end: fuse, and the 22Ω pair

VBUS passes first through **F1**, a **500mA-hold PTC resettable fuse** (a 1206 Littelfuse 1206L050-class part). The math is a hold-vs-trip decision: the 32U4 board draws roughly 25–50mA normally and short of 200mA with every LED lit and a sensor hung off the headers, comfortably under the 500mA hold current; a hard short trips the PTC within milliseconds at an amp or more, and USB itself is only guaranteed to source

500mA — so the fuse also protects the host. After a fault clears, the PTC heals; no soldering iron required.

The data pair gets its own treatment, straight from the datasheet. Section 2.2.8 and 2.2.9 of Atmel-7766J state plainly that D– and D+ “*should be connected to the USB D–/D+ connector pin with a serial 22Ω resistor.*” The resistors damp reflections on the 90Ω differential pair and limit the inrush current that flows into the chip's internal ESD structures when a cable is hot-plugged. Two 0603 22Ω parts (R1, R2) do the job; the layout keeps them close to the connector and the pair length-matched by eye.

22Ω

series resistance the ATmega32U4 datasheet mandates on both D+ and D– (Atmel-7766J §2.2.8/9)

4.2 Decoupling: one capacitor per pin, and why

A TQFP-44 has three VCC pins (14, 24, 34), an AVCC (44), a UVCC (2), and a dedicated UCAP (6) — six supply entrances, each with a different job. The design rule this board follows is the one every professional layout obeys and every beginner skips: **every supply pin gets its own 100nF ceramic, placed within a couple of millimetres of the pad**, before anything else steals the space. A digital output switching 20mA in 5ns is, to the power network, a demand for amps; the local 100nF answers that demand so the demand never propagates. The 10μF pair (C1, C2) at the power entry handles the slower, bulkier events — inrush and the fuse's trip transient.

Pin	Part	Value	Why it exists
14 / 24 / 34	C3 / C4 / C5	100nF	One local decoupler per VCC pin — high-frequency supply for core and I/O
2 (UVCC)	C8	100nF	Decouples the USB pad supply
6 (UCAP)	C7	1μF	Datasheet §2.2.12: output of the internal 3.3V regulator that biases the USB pads
44 (AVCC)	FB1 + C6	bead + 100nF	Analog supply fed through a ferrite bead so digital noise stays out of the ADC
42 (AREF)	C9	100nF	Stabilizes the ADC reference; costs nothing if unused
Rail entry	C1 + C2	2 × 10μF	Bulk energy for USB inrush and load steps

Table 4-1 — The decoupling plan. Every supply entrance has a local capacitor.

The analog branch deserves its own sentence of theory. AVCC powers the ADC's sample-and-hold and reference buffer; if it shares the noisy digital rail directly, that noise appears in every conversion. A ferrite bead (FB1, specified by impedance — roughly 600Ω at 100MHz, in an 0603 body) presents a low resistance at DC but a high impedance to the switching harmonics, and the 100nF at the pin completes a low-pass filter with a corner in the hundreds of kilohertz. The result is a cleaner analog supply for the price of one cent of passive.

Finally, the grounds. Four GND pins (15, 23, 35, 43) plus UGND (5) all tie to the same ground net — on the PCB they should reach a solid, uninterrupted ground pour on the bottom layer. The USB shell tabs are soldered to that same ground for shielding. On a 2-layer board the ground pour is not optional decoration; it is the return path for every signal in Figure 4-1, and cutting it with a careless trace is the classic first-board failure.

5. The Clock: Crystal Theory and Load-Capacitance Math

USB full-speed devices need a $\pm 0.25\%$ (2500ppm) accurate 48MHz bit clock, which the 32U4 synthesizes from its main 16MHz crystal via an internal PLL. The crystal is therefore not a decoration — USB literally does not enumerate without it. This section builds the oscillator circuit the way the datasheet intends it, and does the arithmetic that most tutorials skip.

Inside the chip, XTAL1/XTAL2 connect to an inverting amplifier — the classic **Pierce oscillator**. The crystal is a mechanically resonant element that behaves, electrically, as a series R-L-C branch (the motional arm) in parallel with a small capacitor C_0 (electrode + holder capacitance). The Pierce topology makes the crystal oscillate at the frequency where the motional branch's inductance resonates with the *external* load it sees — and here is the point: **the load is designed, not discovered**.

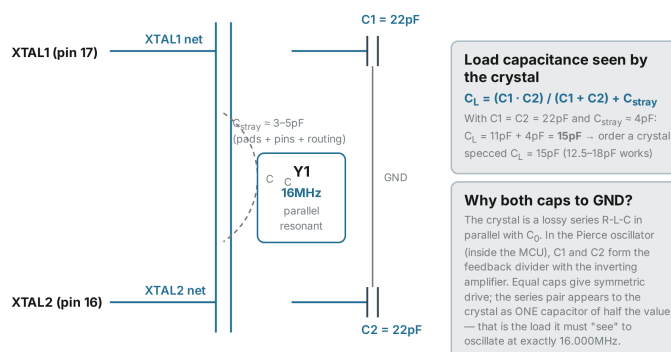


Figure 5-1 — The Pierce oscillator's external world: two equal load capacitors and the stray capacitance of pads and routing. The crystal 'sees' their series combination plus strays.

5.1 The load-capacitance equation, worked

The crystal sees C1 and C2 effectively in *series* (they straddle the crystal to ground), plus the stray capacitance of the two MCU pins, the PCB pads and the traces themselves:

Worked example — the load the crystal must 'see' to hit exactly 16.000MHz

$$CL = (C1 * C2) / (C1 + C2) + C_{stray}$$

With C1 = C2 = 22pF and C_{stray} ~ 4pF (2 pins ~ 2x1.5pF + routing ~ 1pF):

$$\begin{aligned} CL &= (22 * 22) / (22 + 22) + 4 \\ &= 484 / 44 + 4 \\ &= 11\text{pF} + 4\text{pF} \\ &= 15\text{pF} \end{aligned}$$

So the design rule falls out cleanly: **order a 16MHz crystal specified for C_L = 15pF** (anything from 12.5pF to 18pF works with graceful error). Run the same equation in reverse for a different crystal: a common 18pF-load part wants C1 = C2 = 2 × (18 – 4) = 28pF, which the datasheet's recommended range (12–22pF for 8–16MHz crystals, Table 6-3) only just permits at its top end. The de-facto Arduino pairing — an 18pF crystal with 22pF capacitors — loads the crystal to about 14pF and pulls the frequency high by roughly 150–200ppm: mathematically 'wrong', practically irrelevant against a 2500ppm USB budget, and a good instinct-check for when engineering precision matters.

15pF designed load capacitance with 22pF caps	±2500ppm USB full-speed frequency tolerance budget	12–22pF datasheet capacitor window for 8–16MHz crystals
--	---	--

5.2 Layout rules the framework enforces for you

Crystal layout has three classic commandments: keep the loop *short*, keep it *away from switching signals*, and never let other signals cross between the crystal and its capacitors. tscircuit encodes the first and third as actual design rules — any trace touching a crystal is checked against a **10mm maximum length and a zero-via limit**, and violations surface in the DRC panel. When you open this project in tsci dev you will find a handful of first-pass warnings of exactly this kind; cleaning them up by nudging C10/C11 closer to Y1 is one of the deliberate exercises in Chapter 8.

One part-selection note: the project uses a through-hole HC-49/S can because it is nearly impossible to solder badly and costs pennies. A 3225 SMD crystal (e.g. the ABM8 family) saves 8mm of board and is the right choice for production; swap the hc49 footprint string for a four-pad 3225 pattern and the schematic logic is unchanged.

6. Reset, Bootloader and ICSP

A bare chip arrives from the factory blank. It has no Arduino bootloader, no fuse configuration, nothing — and it needs a controlled way to be reset, a controlled way to enter its bootloader, and a port through which the first firmware arrives. This chapter wires all three, and explains the AVR fuse model that governs the boot behaviour.

6.1 The reset network

RESET (pin 13) is active-low with a 10k Ω pull-up (R3) to +5V and a tactile button (SW1) to ground. Pressing SW1 drags the pin low and holds the chip in reset; releasing it lets the internal reset circuit time a clean start-up. The pull-up value is a compromise the datasheet blesses: strong enough (0.5mA when pressed) to hold a solid logic high against noise, weak enough that any ICSP programmer — which drives RESET directly — overpowers it without stress. No capacitor sits on the line: the 32U4 has its own internal reset delay and spike filter, and with a native-USB bootloader there is no DTR auto-reset circuit to accommodate.

6.2 BOOT: the hardware bootloader pin

PE2 (pin 33) doubles as **#HWB** — hardware bootloader entry. The datasheet (§27.5.3) is explicit: with the HWBE fuse programmed, the chip enters its bootloader if HWB is held *low* during an external reset. The board gives PE2 the same 10k Ω pull-up (R4) plus its own button (SW2) to ground. The recipe for a bootloader that refuses to die: **hold BOOT, tap RESET, release BOOT** — the MCU wakes in the bootloader regardless of what the application flash contains. In normal operation the pull-up keeps PE2 a free GPIO, broken out on header J4.

This matters more than beginners expect. The standard Caterina bootloader (what the Arduino IDE pushes for a Leonardo) starts on every reset, waits ~750ms for an upload, then jumps to the application. If the application crashes instantly, that window can be hard to hit. The HWB button gives you a guaranteed entry path with zero timing skill — one fuse bit and one resistor buys a board you cannot brick by software alone.

6.3 Fuses, bit by bit

AVR 'fuses' are just persistent configuration bytes, programmed over ICSP like flash. Three bytes configure this board; the values below are the standard Caterina set used by every 32U4 Arduino:

Fuse	Value	Bit decode	Effect on this board
lfuse	0xFF	CKSEL=1111, SUT=11, CKDIV8=1	Full-swing crystal oscillator, slow start-up (rock-solid with long traces), no clock divide — 16MHz core
hfuse	0xD8	BOOTRST=0, BOOTSZ=01, EESAVE=0	Reset vector into the 4KB bootloader section; EEPROM not preserved on chip erase
efuse	0xCB	HWBE=0 (programmed)	HWB pin active: bootloader forced when PE2 is low at reset

Table 6-1 — The three AVR fuse bytes. '0' means the bit is programmed (active).

6.4 ICSP: the umbilical port

J3 is the standard 2×3 AVR ISP header — MISO, VCC, SCK, MOSI, RESET, GND — wired straight to PB3, PB1, PB2 and RESET. Any USBasp, AVRISP mkII, or a \$3 Arduino programmed as 'Arduino as ISP' can drive it. Because the 32U4's SPI pins are shared with the D14–D16 header positions, the same wires that flash the bootloader later become a user SPI port; that sharing is intentional and standard.

First contact with a blank chip

```
# One-time bootloader + fuse install (USBasp example):
avrdude -c usbasp -p atmega32u4 \
  -U flash:w:caterina-leonardo.hex:i \
  -U lfuse:w:0xFF:m -U hfuse:w:0xD8:m -U efuse:w:0xCB:m

# Read back to verify:
avrdude -c usbasp -p atmega32u4 -U lfuse:r:-:h -U hfuse:r:-:h -U efuse:r:-:h
```

7. Indicators, I²C and the Breakout Headers

7.1 Four LEDs, three calculations

Every LED on the board is an Ohm's-law exercise with real constraints: USB budgets 500mA, modern LEDs are blinding above 10mA, and the MCU pin has a 40mA absolute maximum it should never approach. With a 1kΩ series resistor and a typical 2.0V forward drop at these currents, each indicator draws $(5.0 - 2.0) / 1000 = 3\text{mA}$ — bright enough indoors, invisible-by-daylight-proof it is not, and a rounding error against the power budget. All four resistors share one value, which keeps the BOM (and your soldering sanity) simple.

LED	Driver	Wiring	Lights when
PWR (green)	+5V rail	V5 → LED → 1kΩ → GND	board has power
L (amber)	PC7 / D13	PC7 → 1kΩ → LED → GND	pin driven HIGH (Arduino convention)
TX (amber)	PD5	V5 → LED → 1kΩ → PD5	pin driven LOW — USB transmit
RX (amber)	PB0 / D17	V5 → LED → 1kΩ → PB0	pin driven LOW — USB receive

Table 7-1 — The indicator set. Note the two opposite polarities — that is not a typo.

The TX/RX wiring looks backwards until you meet the Arduino core. The 32U4 has no dedicated LED-mux pins, so the Leonardo's firmware drives activity LEDs in software: the variant file defines TXLED1 as “drive PD5 low” and RXLED1 as “drive PB0 low”. Wiring the LEDs from +5V *into* the pins makes 'low' mean 'lit' — so this board plugs into the Arduino Leonardo board definition and the TX/RX LEDs simply work, blinking on USB traffic with no user code. Reproduce the polarity exactly if you want that compatibility.

3mA

per LED with the chosen 1k Ω / 2.0V combination — (5.0 – 2.0V) / 1k Ω , four times over

7.2 I²C pull-ups

SDA (PD1/D2) and SCL (PD0/D3) carry 4.7k Ω pull-ups (R9, R10) to +5V, so any I²C sensor works the moment it is plugged in. The value is the standard bus-engineering answer: strong enough to charge typical bus capacitance (100pF-class) at 100kHz and 400kHz with clean rise times, weak enough that a 1mA logic-low budget is respected and the pins remain perfectly usable as plain GPIO when you are not running a bus. If you hang a sensor module that carries its own pull-ups, the parallel combination simply stiffens the bus slightly — rarely a problem at these currents.

7.3 The headers

J2 and J4 are 1×16 male headers on 2.54mm pitch, one per board edge, breadboard-ready and labelled on the silkscreen in Leonardo order. The names are not cosmetic: with the Caterina bootloader installed, you select 'Arduino Leonardo' in the IDE and every pin number in every tutorial on the internet addresses the right physical pin on your board.

J2 pin	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Net	V5	GN D	D1 3	D1 2	D1 1	D1 0	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0

Table 7-2 — J2 (digital edge): power at the USB end, then D13 down to D0.

J4 pin	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Net	V5	GN D	AR EF	A0	A1	A2	A3	A4	A5	SC L	SD A	MI SO	SC K	MO SI	SS	H WB

Table 7-3 — J4 (analog edge): analog, I²C and SPI, plus the boot pin.

8. PCB Layout and Routing Craft

The jump from netlist to board is where electronics becomes a physical craft. This chapter walks the layout decisions encoded in `index.tsx`, then hands you the autorouter's honest first-pass report and the mental model for cleaning it up — because reading a DRC list *is* the job.

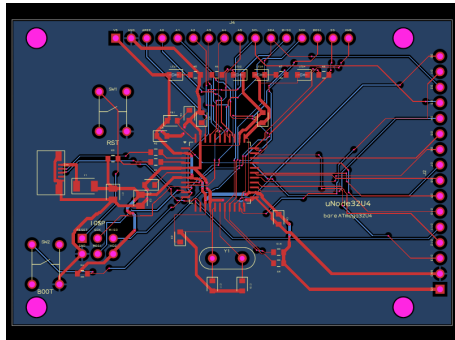


Figure 8-1 — The 2-layer PCB as generated from the code: USB at the left edge feeding the MCU's USB pins directly, crystal below, indicator row above the digital header, buttons lower-left, ICSP right, mounting holes at the corners.

8.1 Placement strategy

Placement follows the signal flow of Figure 1-1. The USB connector sits on the left edge because pins 2–7 of the TQFP are on the upper-*left* edge — the datasheet drew the board map for you. The crystal and its two 22pF caps occupy the quiet zone directly below the chip, as far from the USB pair and the switching GPIO banks as the board allows. The four decoupling caps for VCC/AVCC line the top edge above their pins; bulk caps live at the power entry. Indicators form a row above the bottom header where they are visible in use; buttons sit lower-left with silkscreen labels; the ICSP header anchors the right edge beside the analog header. Nothing is placed because it looked tidy — every zone is a rule about return currents or reach.

8.2 The routing pass — and its honest report

The included verification toolchain renders the board headlessly and checks itself three ways. **npm run verify** audits the structure: all 35 components placed without collisions, 15 named nets formed, every port connected except five deliberately floating pins (the USB ID pin and the internally mirrored button contacts),

and **105 of 105 traces routed**. **npm run proof** machine-checks 90 electrical assertions derived from the datasheet and the Arduino Leonardo variant — pinout, USB front-end, decoupling, clock, reset, ICSP grid, pin map, indicator wiring. And **tsci build** runs the framework's own DRC. The current build passes all three with **zero errors**:

DRC item	Count	What it means	How to clean it
DRC errors (tsci build)	0	Every trace clears every pad and trace by the 0.1mm minimum	Nothing to do — verified
Crystal rule violations	0	All XTAL traces within 10mm, zero vias, framework-enforced	Nothing to do — verified
Electrical proof failures	0	90/90 datasheet-derived assertions pass (npm run proof)	Nothing to do — verified
Unrouted nets	0	105/105 traces routed by the autorouter + hand-routed power lanes	Nothing to do — verified

Table 8-1 — The first-pass DRC report, mapped to fixes.

105/105 traces routed, zero unresolved connectivity	0 placement overlaps after layout iteration	90/90 electrical proof assertions passing (npm run proof)
---	---	---

8.3 Layout rules worth internalizing

- **D+/D− as a pair.** Route them together, same length (a few mm of mismatch is harmless at full-speed), and never split them around the crystal or the buck of a future regulator.
- **Crystal keep-out.** No fast signals under or beside Y1; its loop (XTAL → crystal → caps → GND) should be the smallest loop on the board.
- **Decouplers at the pin, not 'near'.** A 100nF that is 10mm from its VCC pin is a decoration; the loop area is the whole point.
- **Ground pour, then don't cut it.** Bottom layer is one continuous GND pour; route signals on top where possible, and via down rather than carve long top-layer detours.
- **USB shell to ground.** Both mounting tabs are soldered to GND — shielding and mechanical retention in one move.

8.4 From code to factory

Manufacturing is one command away: `tsci export` writes gerbers, a pick-and-place file and a BOM. For JLCPCB-class fabs order 1.6mm FR-4, 2 layers, HASL or ENIG surface finish, and 1oz copper; every footprint used here sits comfortably inside their standard capability (0.3mm pads on the USB connector and 0.8mm TQFP pitch are routine). Order the stencil alongside the boards — the TQFP and 0603s are far happier under a stencil-applied paste and a hot plate than under an iron, though Chapter 10 includes a hand-soldering path too.

9. Bill of Materials

The full BOM below totals roughly ten US dollars of parts at single-quantity pricing, dominated by the microcontroller itself. Package codes map directly to the footprint strings in the code, so every line is also a lookup key into `index.tsx`. Supplier stock codes drift — search the MPN at your distributor of choice rather than trusting any cached part number, including ones in this table.

Ref	Part	Package / footprint	Qty	Notes
U1	ATmega32U4-AU (Microchip/Atmel)	TQFP-44 (tqfp44)	1	The bare MCU; AUR = tape&reel; AU = tray
J1	USB micro-B receptacle, 5-pin SMD	custom (lib/usb-micro-b.tsx)	1	Generic 7.4mm-body part; verify pad map against the datasheet of the part you buy
J2, J4	Pin header, male 1×16, 2.54mm	pinrow (pinheader)	2	Or female sockets if you prefer
J3	Pin header, male 2×3, 2.54mm	pinheader doubleRow	1	AVR ICSP standard order
Y1	16MHz crystal, CL = 15pF (12.5–18pF ok)	HC-49/S (hc49)	1	Swap to a 3225 SMD part + footprint if you like
F1	PTC resettable fuse, 500mA hold	1206	1	Littelfuse 1206L050 class
FB1	Ferrite bead, ~600Ω @ 100MHz	0603	1	Any DC-resistance-low bead
SW1, SW2	Tactile switch, 6×6mm through-hole	pushbutton	2	Standard 4-pin momentary

Table 9-1 — Semiconductors, connectors and electromechanical parts.

Ref	Value	Package	Qty	Role
C1, C2	10 μ F X5R 16V	0805	2	Bulk at power entry
C3–C6, C8, C9	100nF X7R 16V	0603	6	Per-pin decoupling (VCC $\times$ 3, AVCC, UVCC, AREF)
C7	1 μ F X5R 16V	0603	1	UCAP regulator output
C10, C11	22pF C0G/NP0 16V	0603	2	Crystal load caps — C0G dielectric for stability
R1, R2	22 Ω 1%	0603	2	USB D+/D– series
R3, R4	10k Ω 1%	0603	2	RESET and HWB pull-ups
R5–R8	1k Ω 1%	0603	4	LED series
R9, R10	4.7k Ω 1%	0603	2	I ² C pull-ups
LED1	Green, 2.0V	led0603	1	Power
LED2–LED4	Amber, 2.0V	led0603	3	L / TX / RX
H1–H4	M3 mounting hole, 3.2mm	hole	4	Unplated

Table 9-2 — Passives. All resistors 1% (same price as 5% today).

10. Bring-Up, First Firmware and Troubleshooting

10.1 Assembly order

Solder in dependency order, testing as you go — the discipline that makes debugging tractable. (1) **U1 first:** flux generously, drag-solder or hot-air the TQFP, then inspect every pin under magnification and flux-clean; a bridged pin is a five-second fix now and a mystery later. (2) Power chain: F1, FB1, then all capacitors — and before the USB connector goes on, ohm out V5-to-GND: it must read as a capacitor charging (rising resistance), never a short. (3) USB connector, crystal, resistors, buttons, LEDs, headers last.

10.2 First power

Plug into a USB port — ideally through a current-metering adapter — and watch the PWR LED. A blank 32U4 board should draw well under 30mA and enumerate as *nothing at all*: a bare chip with no firmware is silent, and that silence is correct. More than ~100mA or a warm chip means stop and check for solder bridges; the troubleshooting table below maps the usual suspects.

10.3 Bootloader and first sketch

Wire your ISP programmer to J3 (pin 1 is MISO, marked on silkscreen) and run the avrdude command from Chapter 6. On success, replug USB: after a moment the OS grows a serial port, and TX/RX LEDs flicker on the Caterina handshake. In the Arduino IDE select **Tools → Board → Arduino Leonardo** and the new port, then blink:

blink.ino — the first program your bare chip runs

```
void setup() {
  pinMode(13, OUTPUT);          // the 'L' LED on J2's D13 neighbor
}
void loop() {
  digitalWrite(13, HIGH); delay(500);
  digitalWrite(13, LOW);  delay(500);
}
```

Upload, and the L LED breathes. From here the board is a Leonardo in every software sense: Serial talks over USB, Keyboard/Mouse libraries work, analogRead() reaches A0–A5 on J4, and Wire.begin() finds SDA/SCL already pulled up. A natural first project is an I²C sensor scan on J4 — two jumpers and ten lines of code exercise the pull-up design of Chapter 7 directly.

10.4 Troubleshooting

Symptom	Likely cause	Fix
PWR LED dark, 0mA drawn	F1 or USB pads unsoldered; shell tabs only	Reflow J1 and F1; check 5V at pin 14
PWR LED dark, fuse hot	Short on V5	Ohm the rail; look under U1 first
Current above 100mA, chip warm	Solder bridge on TQFP pins, reversed LED	Magnifier pass on all 44 pins
avrdude: 'target not answering'	RST wiring, wrong -p flag, or SW1 stuck	Verify .J3 > .RESET continuity; use -p atmega32u4
avrdude: 'invalid device signature'	Bridged ISP lines or MCU oscillator dead	Check Y1/C10/C11 solder; try -F never blindly
Port appears, upload times out	Caterina window missed	Hold BOOT (SW2), tap RESET, release BOOT, upload immediately
No TX/RX blink	LEDs backwards (must be V5 → LED → R → pin)	Check polarity against Table 7-1
USB 'device descriptor error'	Crystal not oscillating (bad load caps or solder)	Inspect C10/C11/Y1; scope XTAL1 if available
L LED never lights	D13 trace or R6/LED2 polarity	Continuity PC7 (pin 32) → R6 → LED2
ADC readings noisy	FB1/AVCC path or AREF cap missing	Verify AVCC = 5V at pin 44; fit C9

Table 10-1 — The bring-up fault tree, ordered by probability.

10.5 Where to go next

- **Add a sensor:** hang a BME280 or OLED on J4's SDA/SCL — the pull-ups are already waiting.

- **Go portable:** the PTC + VBUS design tolerates a battery + regulator feeding V5; add a diode-OR or a proper power-path IC as the next design exercise.
- **Learn sleep:** the 32U4 idles at microamps; interrupt-driven wake from PE6 (SW-shared D7) is a lesson in AVR power registers.
- **Version two:** shrink the crystal to 3225, add a USBLC6 ESD array, pour stitched ground vias — then publish the board to the tscircuit registry as your first reusable part.